



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

PROYECTO DE BASE DE DATOS II

SafeBridge: Orquestador Multi-Motor de RespalDOS y Validación de Integridad

Documentación Técnica y Manual de Usuario

FD05

CURSO	Base de Datos II
DOCENTE	Ing. Patrick José Cuadros Quiroga
EMPRESA / EQUIPO	BitCraft Solutions
INTEGRANTE	Sierra Ruiz, Iker Alberto (2023077090)
INTEGRANTE	Cortez Mamani, Julio Samuel (2023077283)

*Sistema *SafeBridge**

Documentación Técnica (DocFX) y Manual de Usuario

VERSIÓN 2.0

Control de versiones

Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	IASR / JSCM	Ing. P. Cuadros	Ing. P. Cuadros	11/06/2026	Versión Original
2.0	IASR / JSCM	Ing. P. Cuadros	Ing. P. Cuadros	04/07/2026	Revisión completa con API ampliada

Índice General

- 1. Documentación Técnica de la API
 - 1.1. Módulo: crypto.rs
 - 1.2. Módulo: backup.rs
 - 1.3. Módulo: connections.rs
 - 1.4. Módulo: logs.rs
 - 1.5. Módulo: db.rs
 - 1.6. Módulo: docker.rs
- 2. Manual de Usuario
 - 2.1. Panel Principal (Dashboard)
 - 2.2. Gestión de Conexiones
 - 2.3. Generación de Respaldos
 - 2.4. Historial y Auditoría
- 3. Estructura del Proyecto
- 4. Dependencias del Proyecto
 - Frontend (Node.js)
 - Backend (Rust)

1. Documentación Técnica de la API

1.1. Módulo: `crypto.rs`

Provee utilidades criptográficas para asegurar las credenciales en almacenamiento local.

ENCRYPT_PASSWORD(PASSWORD: &STR) -> RESULT<STRING, STRING>

Cifra una contraseña usando AES-256-GCM con nonce aleatorio de 12 bytes.

- **Entrada:** `password` — Contraseña en texto plano.
- **Salida:** Cadena Base64 que contiene `nonce (12 bytes) + ciphertext`.
- **Error:** Si el cifrado falla internamente.

DECRYPT_PASSWORD(ENCRYPTED: &STR) -> RESULT<STRING, STRING>

Descifra una contraseña previamente cifrada.

- **Entrada:** `encrypted` — Cadena Base64 de nonce+ciphertext.
- **Salida:** Contraseña en texto plano.
- **Error:** Si la longitud es < 12 bytes o el descifrado falla.

1.2. Módulo: `backup.rs`

Orquestador principal de subprocesos y analizador de archivos.

GENERATE_BACKUP(APP, STATE, CONNECTION_ID) -> RESULT<BACKUPRESULT, STRING>

Comando Tauri principal. Orquesta todo el ciclo de backup.

- **Flujo:** Consulta conexión → Descifra password → Ejecuta sidecar → Calcula SHA-256 → Verifica EOF → Inserta log.
- **Retorna:** `BackupResult { file_path, size_bytes, sha256, verified }`.

VERIFY_BACKUP(APP, CONN, FILE_PATH, PASSWORD, LOG_BUFFER) -> RESULT<BOOL, STRING>

Lee los últimos 256 bytes del archivo para buscar firmas de finalización.

- **MySQL:** Busca `"Dump completed on"`.
- **PostgreSQL:** Busca `"PostgreSQL database dump complete"`.
- **SQL Server / MongoDB:** Valida que tamaño > 0.

CALCULATE_HASH_AND_SIZE(PATH) -> RESULT<(U64, STRING), STRING>

Calcula SHA-256 por chunks de 8KB sin cargar el archivo completo en RAM.

- **Retorna:** Tupla `(size_bytes, sha256_hex_string)`.

1.3. Módulo: `connections.rs`

CRUD de conexiones con cifrado integrado.

CREATE_CONNECTION(STATE, CONN) -> RESULT<STRING, STRING>

Genera UUID v4, cifra la contraseña e inserta en SQLite. Retorna el UUID.

LIST_CONNECTIONS(STATE) -> RESULT<VEC<CONNECTIONINFO>, STRING>

Lista todas las conexiones. El campo `password` siempre es `None` (no se envía al frontend).

UPDATE_CONNECTION(STATE, ID, CONN) -> RESULT<(), STRING>

Actualiza una conexión. Si se envía contraseña no vacía, se re-cifra. Si está vacía, se mantiene la anterior.

DELETE_CONNECTION(STATE, ID) -> RESULT<(), STRING>

Elimina la conexión. Los logs asociados mantienen `connection_id = NULL` por ON DELETE SET NULL.

TEST_CONNECTION(HOST, PORT) -> RESULT<BOOL, STRING>

Abre un socket TCP con timeout de 3 segundos usando `TcpStream::connect_timeout`.

1.4. Módulo: `logs.rs`**LIST_LOGS(STATE, ENGINE, STATUS) -> RESULT<VEC<BACKUPLOG>, STRING>**

Lista logs con filtros opcionales por motor y estado. Ignora filtros si son `"all"` o vacíos.

GET_DASHBOARD_STATS(STATE) -> RESULT<DASHBOARDSTATS, STRING>

Retorna estadísticas agregadas: total conexiones, backups OK, backups FAIL, bytes totales y últimos 5 logs.

1.5. Módulo: `db.rs`**INIT_DB(APP_DIR) -> RESULT<CONNECTION>**

Crea el directorio de la app si no existe, abre/crea `safebridge.db`, habilita `PRAGMA foreign_keys = ON`, crea tablas `connections` y `backup_logs`, y aplica migraciones incrementales.

1.6. Módulo: `docker.rs`**CHECK_DOCKER() -> BOOL**

Ejecuta `docker info` y retorna `true` si Docker está disponible.

2. Manual de Usuario

2.1. Panel Principal (Dashboard)

El **Dashboard** es la pantalla inicial. Provee un vistazo rápido a la salud de sus respaldos.

- **4 Tarjetas KPI:** Total Conexiones, Backups Exitosos, Backups Fallidos, Datos Resguardados.
- **Actividad Reciente:** Últimos 5 backups con estado, motor, fecha y tamaño.
- Si un backup ha fallado, la tarjeta se ilumina en rojo.

2.2. Gestión de Conexiones

1. Navegue a **Conexiones** en el panel lateral.
2. Clic en **"Nueva Conexión"**.
3. Seleccione motor (PostgreSQL, MySQL, SQL Server, MongoDB). El puerto se ajusta automáticamente.
4. Ingrese Host, Puerto, Usuario, Contraseña, Nombre de BD.
5. Seleccione la carpeta destino con el botón de carpeta (diálogo nativo del SO).
6. Presione **Test Connection** para verificar conectividad TCP.
7. Presione **Guardar**. La contraseña se cifra con AES-256-GCM automáticamente.

2.3. Generación de Respaldos

1. Diríjase a **Generar Backup**.
2. Seleccione la conexión del dropdown.
3. Pulse **Iniciar Backup**.
4. **Terminal en Vivo**: Observe el avance en tiempo real con mensajes de info (gris), éxito (verde) y error (rojo).
5. **Barra de progreso**: Muestra la fase actual.
6. Al concluir, un card indica BACKUP VERIFICADO (verde) u OPERACIÓN FALLIDA (rojo).

2.4. Historial y Auditoría

1. Navegue a **Historial**.
2. Use los filtros de Motor y Estado para refinar resultados.
3. Use la barra de búsqueda para encontrar por nombre de conexión.
4. Clic en cualquier fila para ver el detalle completo: - ID de operación, Estado, Conexión, Motor - Tiempos de inicio/fin, Ruta del archivo, Tamaño - Verificación EOF (Exitosa/Fallida) - Mensajes de error (si los hay) - Logs crudos de ejecución (expandible)

3. Estructura del Proyecto

```

proyecto-si783-2026-i-u2-backup-restaurar-bd/
├─ index.html           # Entry HTML para Vite
├─ package.json         # Dependencias Node.js
├─ vite.config.ts       # Configuración Vite + Tauri
├─ tsconfig.json        # TypeScript config
├─ src/                 # Frontend React
│   ├── main.tsx        # Punto de entrada React
│   ├── App.tsx         # Router principal
│   ├── App.css         # Estilos globales
│   ├── components/
│   │   ├── Sidebar.tsx # Navegación lateral
│   │   └── ConnectionForm.tsx # Formulario de conexiones
│   └── pages/
│       ├── Dashboard.tsx # KPIs y actividad reciente
│       ├── Connections.tsx # CRUD de conexiones
│       ├── Backup.tsx    # Orquestador con terminal
│       └── History.tsx   # Historial filtrable
├─ src-tauri/           # Backend Rust
│   ├── Cargo.toml      # Dependencias Rust
│   ├── tauri.conf.json # Configuración Tauri
│   └── src/
│       ├── main.rs     # Entry point Rust
│       ├── lib.rs      # AppState, plugins, handlers
│       ├── models.rs   # Structs: ConnectionInfo, BackupLog
│       ├── db.rs       # Inicialización SQLite
│       ├── crypto.rs   # AES-256-GCM encrypt/decrypt
│       ├── connections.rs # CRUD + test TCP
│       ├── backup.rs   # Orquestador + EOF + SHA-256
│       ├── logs.rs     # Consultas de auditoría
│       └── docker.rs   # Verificación Docker

```

4. Dependencias del Proyecto

Frontend (Node.js)

Paquete	Versión	Propósito
react	^19.1.0	Framework UI
react-dom	^19.1.0	Renderizado DOM
@tauri-apps/api	^2	Comunicación IPC con Rust
@tauri-apps/plugin-dialog	^2.7.1	Diálogos nativos del SO
lucide-react	^1.17.0	Librería de iconos
react-hot-toast	^2.6.0	Notificaciones toast
tailwindcss	^4.3.0	Framework CSS
typescript	~5.8.3	Tipado estático
vite	^7.0.4	Bundler/Dev server

Backend (Rust)

Crate	Versión	Propósito
tauri	2	Framework desktop
tauri-plugin-shell	2	Ejecución de sidecars
tauri-plugin-dialog	2.7.1	Diálogos nativos
rusqlite	0.40.0	SQLite embebido
aes-gcm	0.10.3	Cifrado AES-256-GCM
sha2	0.11.0	Hash SHA-256
uuid	1.23.2	Generación UUID v4
chrono	0.4.44	Manejo de fechas/hora
serde	1	Serialización JSON
base64	0.22.1	Codificación Base64
rand	0.10.1	Generación de nonces aleatorios
hex	0.4.3	Codificación hexadecimal

Documento actualizado por el equipo BitCraft Solutions — Universidad Privada de Tacna, FAING-EPIS, Ciclo 2026-I.